DAY 06 | HANDS-ON ACTIVITIES

FastAPI Core and SQLAlchemy Intro: Problem Statements

Seven problems that build one Books API step by step. Test with TestClient in Colab or /docs in VS Code.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Parameters Playground

Level: Warm-up | **Concepts:** path, query, Query()

Create GET /books/{book_id} and GET /books with optional author and a limit between 1 and 50 (default 10). Try invalid values and note the status codes.

Sample run / expected result

```
GET /books?limit=500 -> 422
GET /books/5 -> {"id": 5}
```

Hints

- limit: int = Query(10, ge=1, le=50)

Problem 2: Book Models

Level: Easy | **Concepts:** Pydantic BaseModel, Field

Define BookCreate (title min 1 char, author, pages > 0, year between 1450 and 2026) and Book (adds id). Create valid and invalid objects in Python and print the errors.

Sample run / expected result

```
BookCreate(title="", ...) -> ValidationError: title too short
```

Hints

- from pydantic import ValidationError
- Wrap creation in try/except ValidationError

Problem 3: Create and List

Level: Easy | **Concepts:** POST body, response_model, 201

Add POST /books (BookCreate in, Book out, 201) and GET /books returning a list of Book. Store books in a dict.

Sample run / expected result

```
POST /books {...} -> 201 {"id": 1, "title": "Ikigai", ...}
```

Hints

- @app.post("/books", response_model=Book, status_code=201)

Problem 4: Read, Update, Delete

Level: Medium | **Concepts:** 404, PATCH, DELETE 204

Add GET /books/{id}, PATCH /books/{id} with a BookUpdate model, and DELETE /books/{id}. Return 404 for unknown ids.

Sample run / expected result

```
PATCH /books/1 {"pages": 250} -> 200 with only pages changed
```

Hints

- data.model_dump(exclude_unset=True)

Problem 5: Use a Router

Level: Medium | **Concepts:** APIRouter, prefix, tags

Move all book endpoints to a router with prefix /books and tag Books. Add a second router for /authors with a simple GET list.

Sample run / expected result

```
/docs shows two groups: Books and Authors
```

Hints

- app.include_router(books_router)

Problem 6: First ORM Model

Level: Challenging | **Concepts:** SQLAlchemy engine, model, session

Create a BookRow model in SQLite. Insert 5 books, query those with more than 200 pages sorted by title, update one, and delete one.

Sample run / expected result

```
Books over 200 pages: Atomic Habits, Deep Work, Ikigai
```

Hints

- select(BookRow).where(BookRow.pages > 200).order_by(BookRow.title)

Problem 7: API + Database

Level: Stretch | **Concepts:** FastAPI + SQLAlchemy

Replace the in-memory dict with SQLAlchemy. Each endpoint opens a Session, does its work and commits. Return rows through Book with from_attributes.

Sample run / expected result

```
POST then GET returns data that survives restarting the app
```

Hints

- model_config = {"from_attributes": True}
- Book.model_validate(row)

Ganbatte (頑張って): do your best